Web & Tech Assignment

Name-Indrani Sahu

Roll number-24B4206

Question 2

Code Snippet

1. Folder Structure

```
project2/
■■■ backend/
■   ■■■ src/
■   ■   ■■■ routes/
■   ■   ■   ■■■ register.js       # Registration API endpoints
■   ■   ■   ■■■ test.js           # Failure scenario test endpoints
■   ■   ■■■ middleware/
■   ■   ■   ■■■ errorHandler.js   # Global error handler
■   ■   ■   ■■■ rateLimiter.js    # Rate limiting middleware
■   ■   ■■■ utils/
■   ■   ■   ■■■ AppError.js       # Custom structured error class
■   ■   ■   ■■■ alerter.js        # Webhook alert system
■   ■   ■■■ tracing.js            # OpenTelemetry setup
■   ■   ■■■ server.js             # Main Express server
■   ■■■ .env                      # Environment variables (secrets)
■   ■■■ .gitignore                # Hides .env from GitHub
■   ■■■ package.json
■■■ frontend/
■   ■■■ src/
■       ■■■ components/
■       ■   ■■■ ErrorBoundary.jsx     # Catches React UI crashes
■       ■   ■■■ RegistrationForm.jsx  # Main registration form
■       ■■■ utils/
■       ■   ■■■ api.js                # Axios with error interceptor
■       ■■■ App.js                    # Root component
■■■ docker-compose.yml                # Runs Grafana monitoring container
```

2. Backend Error Handling

2.1 Custom AppError Class

Instead of throwing generic JavaScript errors, a structured AppError class was created. It carries a statusCode and an isOperational flag. Operational errors show the real message to users. Non-operational errors (bugs) show a generic message to protect internal details.

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}
module.exports = AppError;
```

2.2 Global Error Handler Middleware

A single Express middleware catches ALL errors from ALL routes. It logs full details for developers, sends webhook alerts for 500 errors, and returns appropriate responses to users without exposing sensitive information.

```
const errorHandler = (err, req, res, next) => {
  err.statusCode = err.statusCode || 500;
  console.error({ timestamp: new Date().toISOString(),
    method: req.method, url: req.url,
    statusCode: err.statusCode, stack: err.stack });
  sendAlert(err, req);
  if (!err.isOperational) {
    return res.status(500).json({ success: false,
      message: 'Something went wrong. Our team has been notified.' });
  }
  res.status(err.statusCode).json({ success: false, message: err.message });
};
```

2.3 Rate Limiting

Two rate limiters protect the application:

- General: 100 requests per 15 minutes per IP address
- Registration: Only 5 attempts per hour per IP
- Returns HTTP 429 when limit is exceeded

2.4 Health Check Endpoint

```
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: Math.floor(process.uptime()) + ' seconds',
```

```
    memory: {
      used: Math.round(process.memoryUsage().heapUsed/1024/1024) + ' MB',
    },
    service: 'ecell-backend'
  });
});
```

3. Frontend Error Handling

3.1 React ErrorBoundary

When a React component crashes during rendering, by default the entire page goes blank. The ErrorBoundary component wraps the entire app and catches any crash, showing a friendly fallback UI with a Try Again button instead of a blank white screen.

```
class ErrorBoundary extends React.Component {
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }
  componentDidCatch(error, errorInfo) {
    console.error('Component Error:', error, errorInfo);
  }
  render() {
    if (this.state.hasError) {
      return (
        <div>
          <h2>Something went wrong</h2>
          <button onClick={() => this.setState({ hasError: false })}>
            Try Again
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

3.2 Axios Interceptor

A central Axios instance handles all API errors in one place:

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    const status = error.response?.status;
    if (status === 429) throw new Error('Too many attempts. Please wait.');
    if (status === 503) throw new Error('Service unavailable. Try later.');
    if (!error.response) throw new Error('Cannot connect to server.');
    throw new Error(error.response.data?.message || 'Something went wrong');
  }
);
```

6. Failure Scenarios

Scenario 1 — Server Crash (GET /api/test/crash)

Simulates a real programming bug by accessing a property of null.

• Returns HTTP 500 with generic safe message to user

• Webhook receives instant notification with full error details

• Terminal logs: GET /api/test/crash 500 2ms

Scenario 2 — Database Timeout (GET /api/test/db-timeout)

Simulates a database connection that times out after 3 seconds.

• Returns HTTP 503 after 3 second delay • Webhook receives notification with route and status details

• Terminal logs: GET /api/test/db-timeout 503 3002ms

Scenario 3 — Rate Limit Exceeded

Simulates spam registrations exceeding the 5 per hour limit.

• Returns HTTP 429 after 5th registration attempt • No alert triggered as this is expected behavior

Webhook Alert Proof

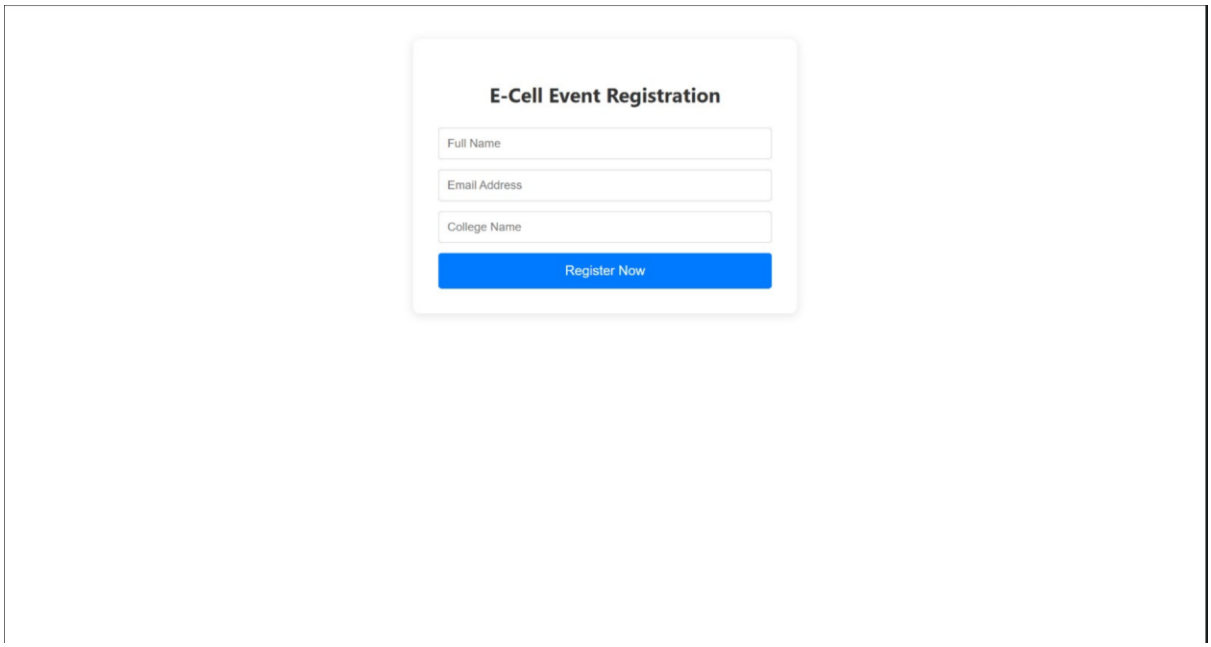
When Scenario 1 triggered, webhook.site instantly received:

```
{
  "title": "ECELL SERVER ERROR",
  "error": "Cannot read properties of null (reading 'someProperty')",
  "route": "GET /api/test/crash",
  "time": "2026-03-08T06:11:10.505Z",
  "statusCode": 500
}
```

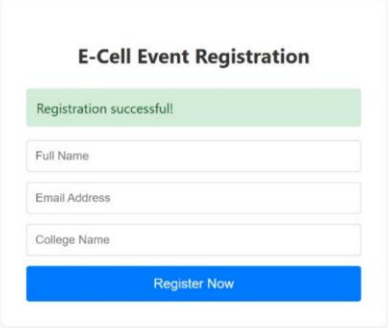
This proves the alert system correctly captures the error message, affected route, exact timestamp, and HTTP status code in real time.

Screenshots of monitoring dashboard

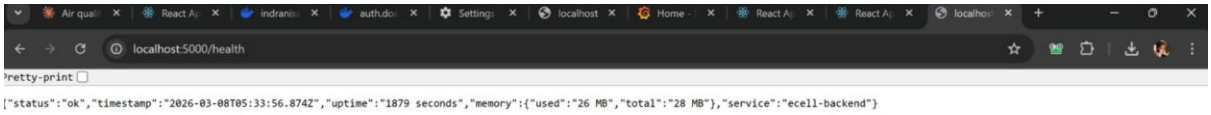
1. Registration Form



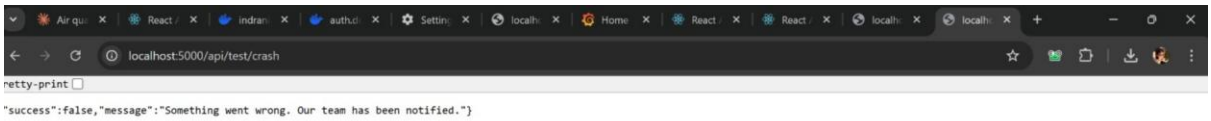
2. Success message after registering



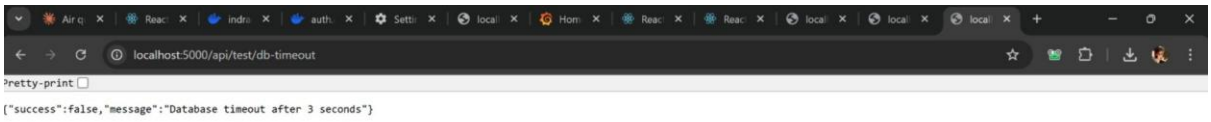
3. /health endpoint showing status ok



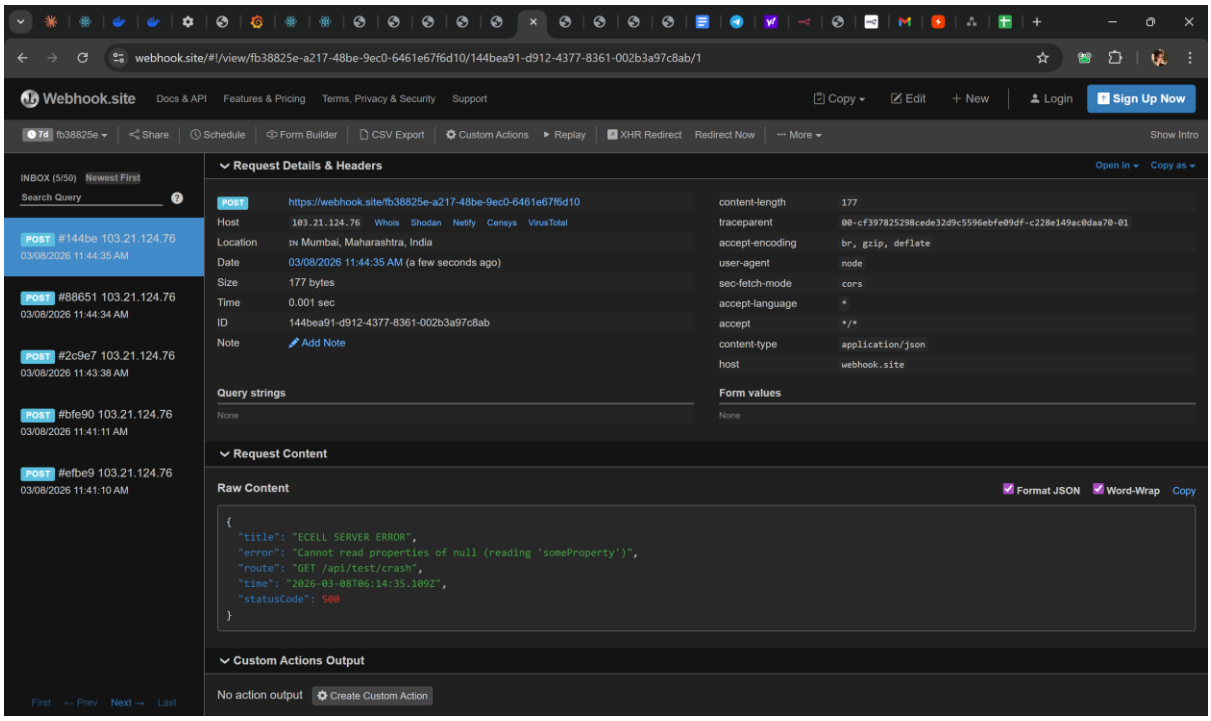
4. /api/test/crash showing 500 error



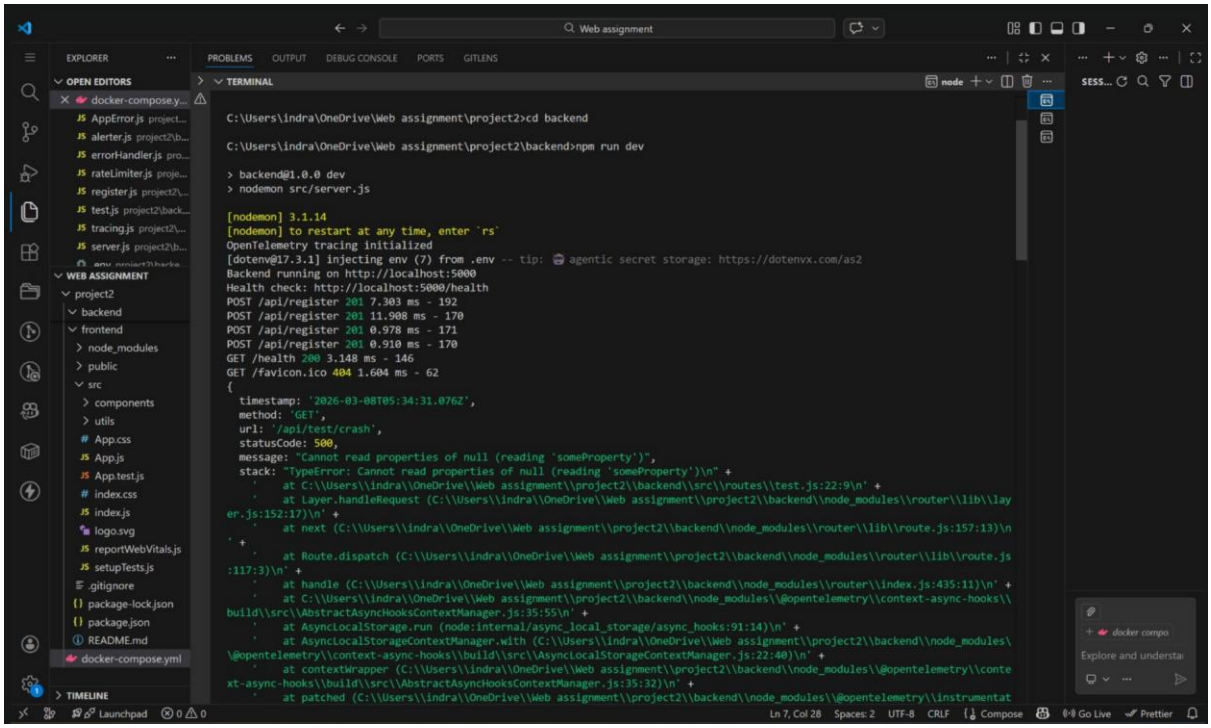
5. /api/test/db-timeout showing 503 error

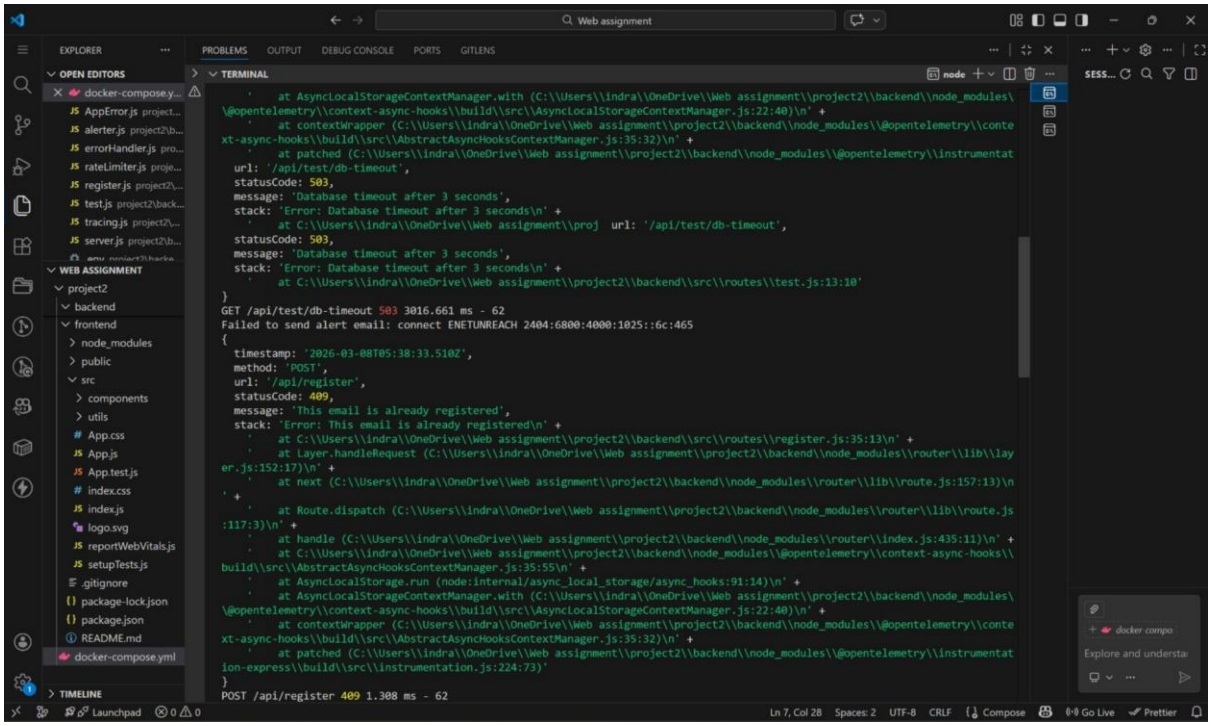


6. Webhook.site showing alert JSON

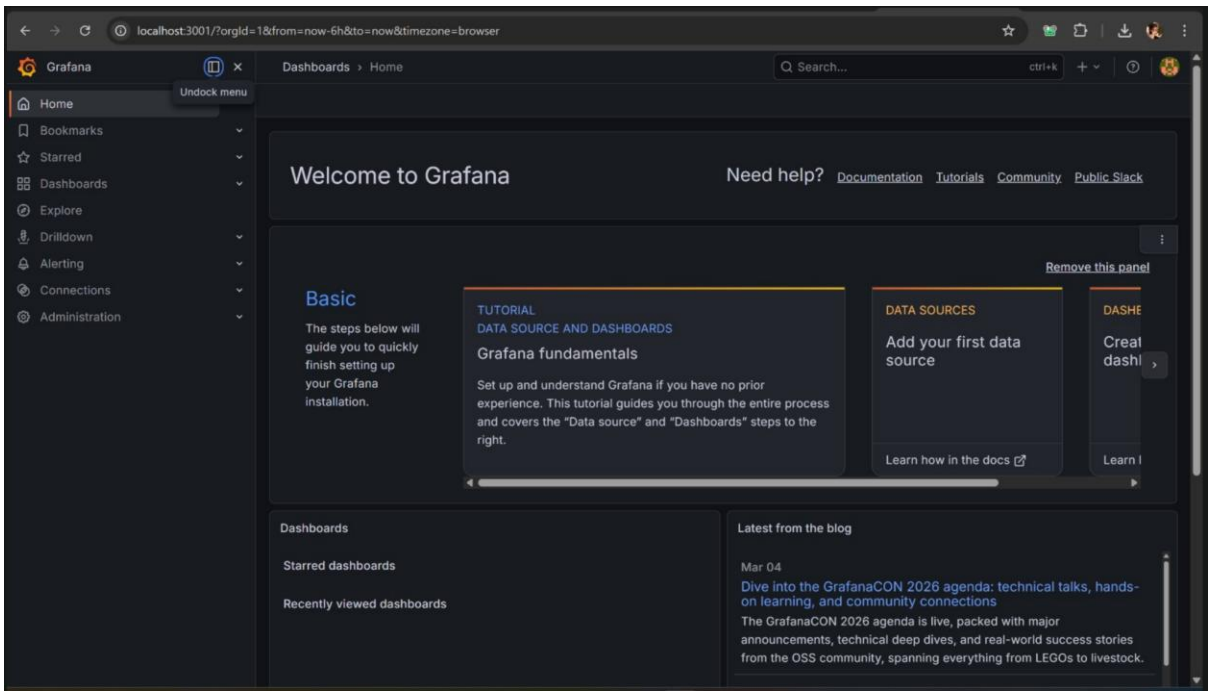


7. Backend terminal showing logs





8. Grafana dashboard



Short explanation of design decisions

1. Backend Error Handling

The backend was built with a **three-layer error handling strategy**. First, a custom AppError class was created to distinguish between two types of errors — operational errors (expected failures like validation errors or duplicate registrations) and programming bugs (unexpected crashes). This separation ensures users always receive clean, helpful messages while developers get full error details in the logs. Second, a single global error handler middleware sits at the end of the Express middleware chain and catches every error from every route automatically — error handling logic is written once and works everywhere. Third, process.on handlers catch any crash that happens outside of routes, ensuring nothing ever goes completely unnoticed.

2. Request Logging

Morgan middleware was integrated to log every incoming request automatically. Each log line captures the HTTP method, route, status code, and response time — for example POST /api/register 201 23ms. This creates a complete audit trail of all server activity, making it easy to trace exactly what happened before an error occurred.

3. Rate Limiting

Two levels of rate limiting were added to protect the system during high-traffic E-Cell events. A general limiter allows 100 requests per 15 minutes per IP address. A stricter registration-specific limiter allows only 5 attempts per hour. When the limit is exceeded, the server returns HTTP 429 immediately, protecting the database from spam and abuse without crashing.

4. Frontend Error Handling

Two separate mechanisms handle errors on the frontend. A React ErrorBoundary component wraps the entire application — if any component crashes during rendering, the user sees a friendly recovery screen with a "Try Again" button instead of a blank white page. A centralized Axios interceptor handles all API communication errors in one place, converting technical HTTP status codes into simple human-readable messages that guide the user on what to do next.

5. Alert System

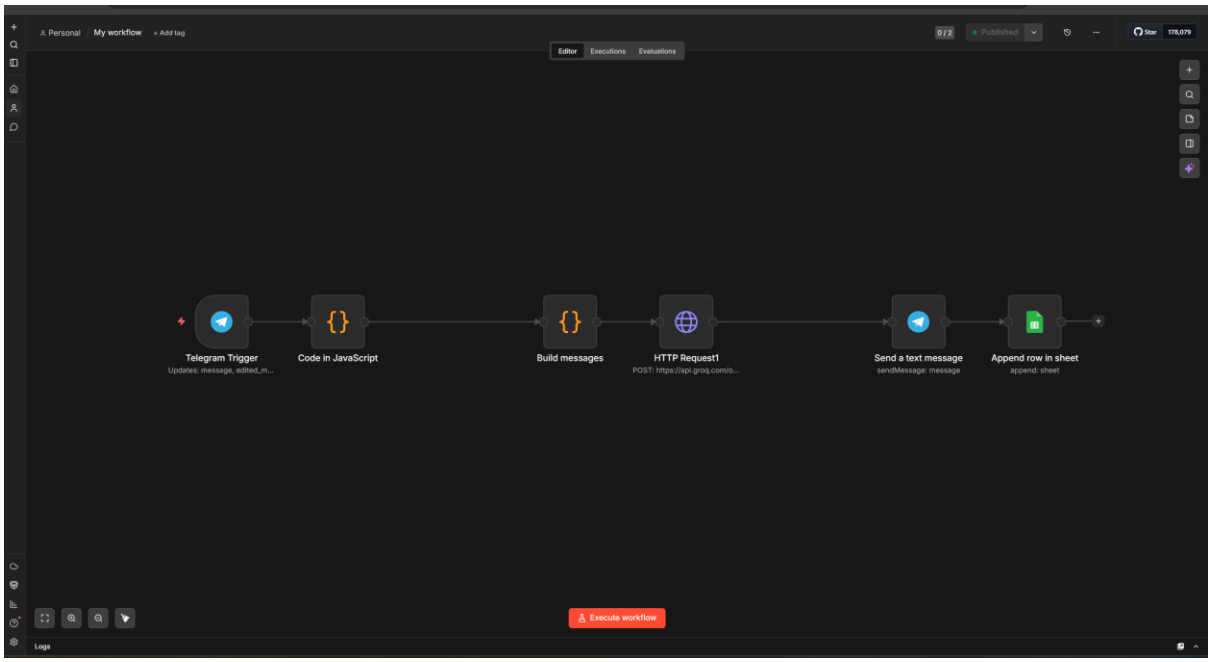
The webhook alert system was chosen over email for a practical reason — SMTP ports (465 and 587) are blocked on most college and office networks, making email delivery unreliable. Webhooks use standard HTTPS port 443 which is always open everywhere. Every time a 500 error occurs, the system automatically sends a real-time alert to webhook.site containing the error message, affected route, timestamp, and status code. In production on a cloud server, email alerts would be added alongside webhooks for complete coverage.

6. Monitoring with Docker

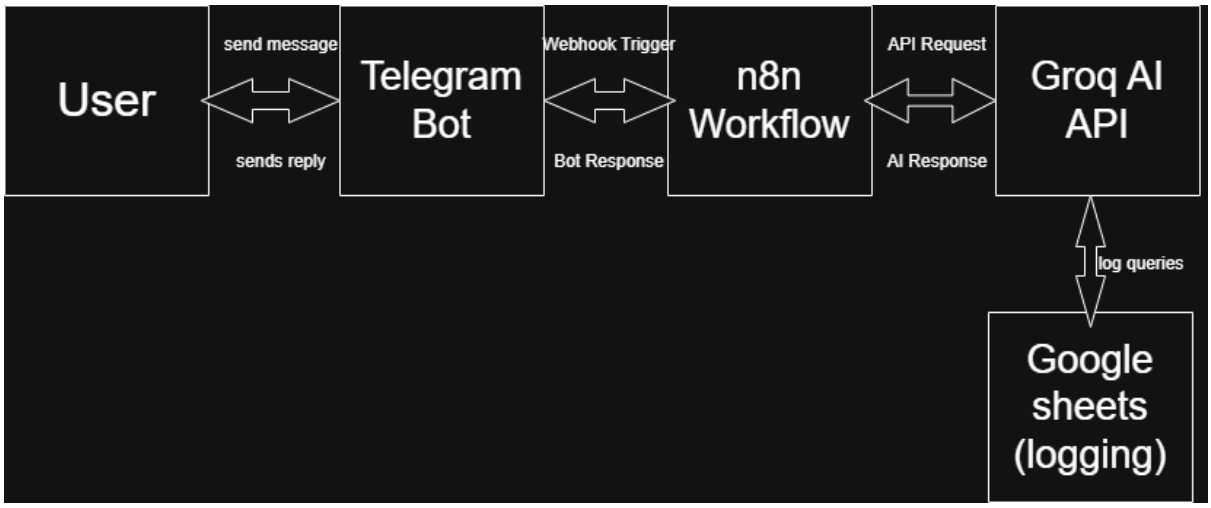
Grafana was containerized using Docker Compose so that any team member can get the full monitoring stack running with a single command — docker compose up -d. This eliminates setup inconsistencies across different developer machines and ensures the monitoring environment is identical in development and production.

Question 3

n8n Workflow diagram



Architecture Diagram



Flow Description:

1. User sends message on Telegram
2. Telegram triggers n8n webhook
3. n8n extracts message and builds context
4. n8n calls Groq AI API for response
5. Groq AI returns intelligent answer
6. n8n sends reply back via Telegram
7. Query and response logged to Google Sheets

Prompt Design Explanation

Prompt Design is the process of writing instructions for an AI model to control how it behaves, what it knows, and how it responds. In our E-Cell chatbot, we designed a System Prompt that tells the Groq AI exactly what role to play and what information to use when answering user questions.

Our System Prompt

The following system prompt was used in the Groq API call:

"You are an official E-Cell event assistant."

Answer questions about:

Events: Startup Summit (March 15), Hackathon (April 2-3),

Business Plan Competition (April 20).

Registration closes 7 days before each event at ecell.org/register.

Certificates issued within 2 weeks via email.

Sponsorship: Gold 1L, Silver 50K, Bronze 25K.

Contact: ecell@college.edu.

Be concise and friendly. Only answer E-Cell related questions.

Prompt Structure - 3 Key Parts

Part 1: Role Definition

Prompt- You are an official E-Cell event assistant

Purpose of Role Definition:

Tells the AI exactly who it is and what job it has

Prevents the AI from behaving like a general assistant

Makes sure all responses stay focused on E-Cell topics

Creates a professional and consistent personality for the bot

Part 2: Knowledge Base

Info-Events, Dates, Registration, Certificates, Sponsorship, Contact

We provided the AI with all necessary E-Cell information:

Category	Information Provided
Events	Startup Summit (March 15), Hackathon (April 2-3), Business Plan Competition (April 20)
Registration	Closes 7 days before each event at ecell.org/register
Certificates	Issued within 2 weeks after event via email
Sponsorship	Gold: ₹1L, Silver: ₹50K, Bronze: ₹25K
Contact	ecell@college.edu

Part 3: Behavior Rules

Rules- Be concise, friendly, only answer E-Cell questions

Rules we set for the AI:

Be concise and friendly - keeps responses short and easy to read

Only answer E-Cell related questions - prevents off-topic responses

Direct unknown queries to ecell@college.edu - handles unanswerable questions

Maintain professional tone - represents E-Cell brand properly

Conversation Context Design

To maintain short conversation context (Task 5), we implemented a memory system using n8n static data:

Last 4 messages are stored per user using their Chat ID

Previous message is passed to Groq API with every new request

This allows the bot to understand follow-up questions

Memory resets after 4 messages to keep responses relevant

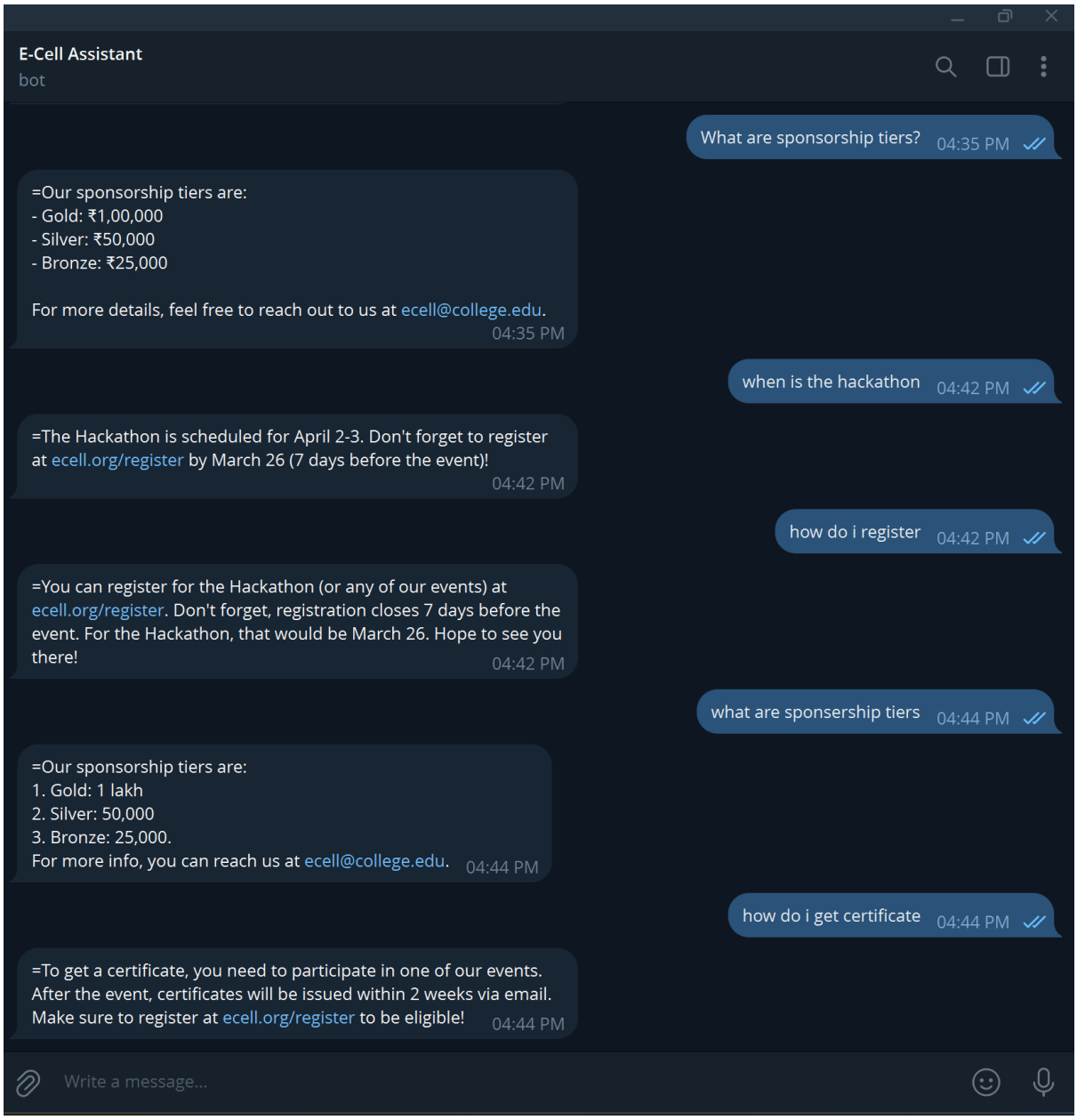
5. Example Conversations

Example 1: Event Query

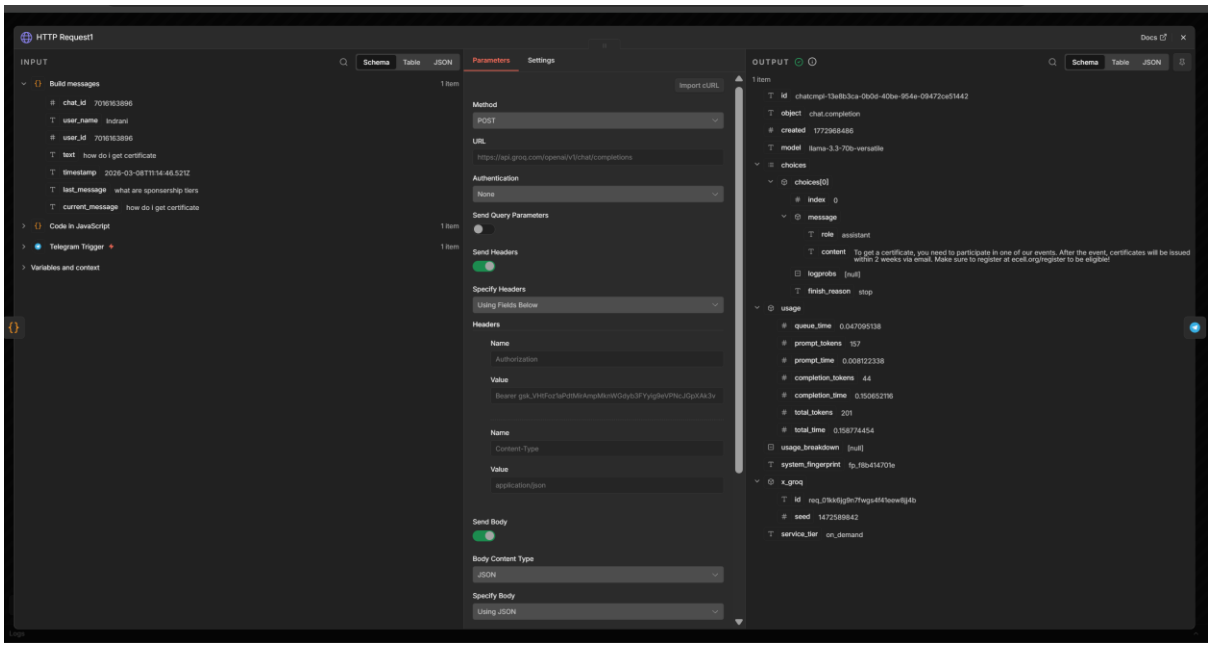
User	When is the hackathon?

Bot	The E-Cell Hackathon is scheduled for April 2-3! Register at ecell.org/register before March 26th.
User	How do I register for it?
Bot	You can register for the Hackathon at ecell.org/register. Registration closes 7 days before the event!

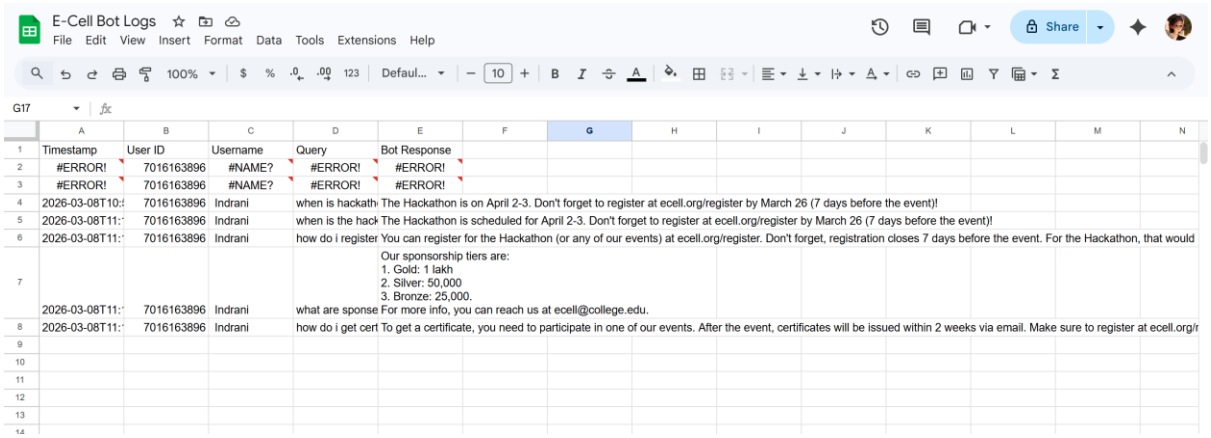
Screenshots of working bot



Screenshot of Groq API Response



Screenshot of Google Sheets logging



E-Cell Bot Logs

File Edit View Insert Format Data Tools Extensions Help

	Timestamp	User ID	Username	Query	Bot Response
1	Timestamp	User ID	Username	Query	Bot Response
2	#ERROR!	7016163896	#NAME?	#ERROR!	#ERROR!
3	#ERROR!	7016163896	#NAME?	#ERROR!	#ERROR!
4	2026-03-08T10::	7016163896	Indrani	when is hackath	The Hackathon is on April 2-3. Don't forget to register at ecell.org/register by March 26 (7 days before the event)!
5	2026-03-08T11::	7016163896	Indrani	when is the hack	The Hackathon is scheduled for April 2-3. Don't forget to register at ecell.org/register by March 26 (7 days before the event)!
6	2026-03-08T11::	7016163896	Indrani	how do i register	You can register for the Hackathon (or any of our events) at ecell.org/register. Don't forget, registration closes 7 days before the event. For the Hackathon, that would
7	2026-03-08T11::	7016163896	Indrani	what are sponse	Our sponsorship tiers are: 1. Gold: 1 lakh 2. Silver: 50,000 3. Bronze: 25,000. For more info, you can reach us at ecell@college.edu.
8	2026-03-08T11::	7016163896	Indrani	how do i get cert	To get a certificate, you need to participate in one of our events. After the event, certificates will be issued within 2 weeks via email. Make sure to register at ecell.org/r

Question 4

Initiative —Problem Statement Platform

The Problem

Every ideathon and startup competition in E-Cell follows the same pattern, students come up with a problem, build a solution around it, and pitch to investors. The result is that most teams spend weeks solving problems that don't really exist, or exist in a form very different from what they imagined. The feedback from investors is almost always the same: *your problem isn't real, or the market doesn't work this way.*

The irony is that the investors and companies sitting in those judging panels carry real, unsolved problems they'd genuinely want student teams to work on. But there is no structured way for them to put those problems in front of student builders. So both sides lose students waste effort on imaginary problems, and industry partners leave without the fresh perspectives they actually came for.

Proposed Solution

A two-part initiative, a web platform that collects real problem statements from industry before the event, and a structured event where students present their solutions to those problems.

Industry partners - VCs, founders, corporate innovation teams submit their problem statements through a web portal in the weeks before the event. Each submission follows a structured format: the problem, the domain, the context, and what a good solution might look like. Students browse these on the platform, choose the problem that interests them, form teams, and spend time building a solution.

On event day, industry partners attend as the audience and judging panel. They watch students present solutions not just to their own problem, but potentially to others as well. The best solutions get recognised, and the teams behind them get direct access to the people who submitted the problem.

The platform is what makes this more than a one-day event. It handles problem submission, student browsing, team formation, sprint-mode progress tracking, final submissions, and digital judging all in one place.

Implementation

Six weeks before the event, the platform goes live with two interfaces. The first is a submission portal for industry partners a clean form where they enter the problem title, domain, context, and what they'd consider a strong solution. The second is a browsing interface for students filterable by domain, difficulty, and company type where they can register interest and find teammates working on the same problem.

In the two weeks before the event, the platform shifts into team formation mode. Students post their skills and the problem they are pursuing. Others can join them. By the time the event starts, teams are already formed and have had time to think.

During the event sprint, teams log progress through the platform. Final submissions deck, prototype link, or demo video are uploaded before the presentation deadline. Judges score through a structured digital rubric on the platform. Results are calculated automatically.

After the event, every industry partner receives an automated summary of all solutions built for their problem. Top teams are connected directly to the relevant company through the platform. All submissions are archived in a public-facing repository, browsable by future students and partners.

Impact

Students stop building in a vacuum. They work on problems with real industry context, which makes the solutions sharper and the learning deeper. They leave the event with a team, a prototype they can continue working on, and a direct connection to an industry partner not just a certificate.

For industry partners, they get to see real student teams tackle their actual problems, and they might walk away with a working prototype built in 48 hours at no cost.

For E-Cell, it will create a living database of industry problems and student solutions that compounds in value every year. Strong sponsorship hook, companies pay to have their problem statement featured and get first access to the top teams.

Long-term Vision

In its first year, this is a focused event with a supporting platform. The real shift happens when the platform stays open year-round.

Industry partners submit problems anytime. Students pick them up as side projects, hackathon entries, or final year thesis topics. The repository grows into a living archive every problem ever submitted, every solution ever built, all searchable. Recruiters browse it to find students who built something real. Investors browse it to spot early-stage ideas worth pursuing.

Over time, E-Cell stops being an event organiser and becomes something closer to an innovation platform — a permanent, structured connection between industry problems and student builders. That is not something you can replicate with a single event. It requires the infrastructure, and building that infrastructure is the point.